2.1

CIPHERMESH

Concurrent Task Daemonization using Threads and Workers

Assignment 2.1 - Detailed Technical Report

A continuation of the Multi-Cloud LLM Privacy and Secret-Protection Agent

Kumar Waibhav Akshat
Register No.: RA2212704010027
M.Tech Integrated - CSE Data Science | AG1
Department of Data Science and Business Systems
College of Engineering and Technology
SRM Institute of Science and Technology

Course: 21CSE525T - Agent Technology
Faculty: Dr. Prabhu Kavin B

Submission: Assignment 2.1

Executive Summary

CipherMesh began in Assignment 1.1/1.2 as a deterministic simple-reflex privacy agent that inspects an LLM request, detects secrets or sensitive data, applies a highest-risk-wins policy, and selects ALLOW, REDACT, REVIEW, or BLOCK. Assignment 2.1 extends that exact security core rather than replacing it. The new engineering problem is execution: a privacy gateway must continue receiving requests, monitoring itself, and writing audit evidence while other requests may be waiting on provider or network I/O.

The implemented solution introduces a bounded request queue, four configurable request-worker threads, a daemon monitor, a separate asynchronous audit writer, lock-protected runtime state, cooperative shutdown events, and sentinel-based worker termination. The primary thread only parses/validates and enqueues work; it is therefore not locked behind each request. The security policy from Assignment 1.2 remains unchanged.

> **Verified outcome**
> The demonstration processed 13 accepted requests with 4 workers and a 120 ms simulated provider I/O delay in about 503.7 ms total wall time. Outcomes were 4 ALLOW, 3 REDACT, 2 REVIEW, and 4 BLOCK, with one unsupported-provider envelope rejected before queue insertion. Five automated tests passed.

Contents

- 1. Continuity from Assignment 1.1 and 1.2
- 2. Assignment 2.1 Requirements and Design Interpretation
- 3. Proposed and Implemented Concurrent Architecture
- 4. Multi-threading / Worker Infrastructure
- 5. Daemon Monitoring Functionality
- 6. Thread Safety & Concurrency Handling
- 7. Detailed Execution Workflow
- 8. Implementation Structure and Key Code
- 9. Performance and Latency Evaluation
- 10. Verification, Testing and Evidence
- 11. Mapping to Unit-2 Component Patterns
- 12. Security and Privacy Properties
- 13. Limitations and Production Evolution
- 14. Conclusion
- Appendix A - Run Instructions
- Appendix B - Deliverable File Structure
- Appendix C - Verified Log Excerpts
- References

1. Continuity from Assignment 1.1 and 1.2

CipherMesh is a provider-neutral privacy control plane positioned before the LLM provider boundary. The earlier project architecture used the sequence PERCEIVE -> PARSE -> REASON -> ACT -> AUDIT and a highest-risk-wins decision hierarchy BLOCK > REVIEW > REDACT > ALLOW. Assignment 1.2 implemented that as a standard-library Python program with synthetic multi-cloud prompts and no external transmission.

Previous capability	State before 2.1	Assignment 2.1 treatment
Request envelope validation	Implemented for AWS Bedrock, Azure AI, GCP Vertex AI and local Ollama	Preserved; invalid/unsupported envelopes are rejected before the work queue.
Secret / PII detectors	Eight deterministic detectors D1-D8	Preserved as an immutable detector tuple shared read-only by workers.
Decision policy	BLOCK > REVIEW > REDACT > ALLOW	Preserved exactly; worker concurrency must never change policy semantics.
Audit trail	Synchronous console-oriented evidence	Upgraded to a privacy-safe asynchronous JSONL audit writer.
Execution model	Single sequential request loop	Upgraded to bounded queue + configurable worker pool + monitor daemon.
Performance evidence	Single execution log	Expanded to per-request queue/service/e2e latency, daemon snapshots, and sequential-vs-threaded benchmarks.

> **Design principle**
> Assignment 2.1 is an infrastructure evolution, not a policy rewrite. This keeps experimental continuity: any change in throughput or responsiveness can be attributed to concurrency rather than altered privacy rules.

2. Assignment 2.1 Requirements and Design Interpretation

The assignment objective is to implement background processing so that the agent can execute background operations without locking the user interface or core event loop. For CipherMesh, the relevant background work is not WiFi polling or travel-deal scanning; it is continuous gateway health/latency monitoring and asynchronous privacy-safe audit persistence while request workers independently inspect prompts.

Assignment requirement	CipherMesh implementation	Evidence
Multi-threading / worker infrastructure	Configurable pool of non-daemon request workers consuming a bounded queue.	ciphermesh_concurrent_agent.py; console_execution.log
Background daemon monitor	MonitorDaemon samples queue depth, worker liveness, decision counts, throughput and latency on a periodic interval.	logs/monitor_metrics.jsonl
Safe concurrent variable sharing	queue.Queue for message passing; threading.Lock for shared metrics; Event for stop signaling; immutable policy data.	RuntimeState, queues, MonitorDaemon
No UI/core lockup	Main thread enqueues quickly and emits CORE_EVENT_LOOP=RESPONSIVE while workers continue.	console_execution.log

Assignment requirement	CipherMesh implementation	Evidence
Execution latency performance logs	Queue-wait, service and end-to-end latency per request plus benchmark CSV/JSON.	execution_latency.log; benchmark_results.csv
Structured component pattern	Worker pool, monitor, audit writer and state are separate lifecycle-managed components.	Architecture and lifecycle sections

3. Proposed and Implemented Concurrent Architecture

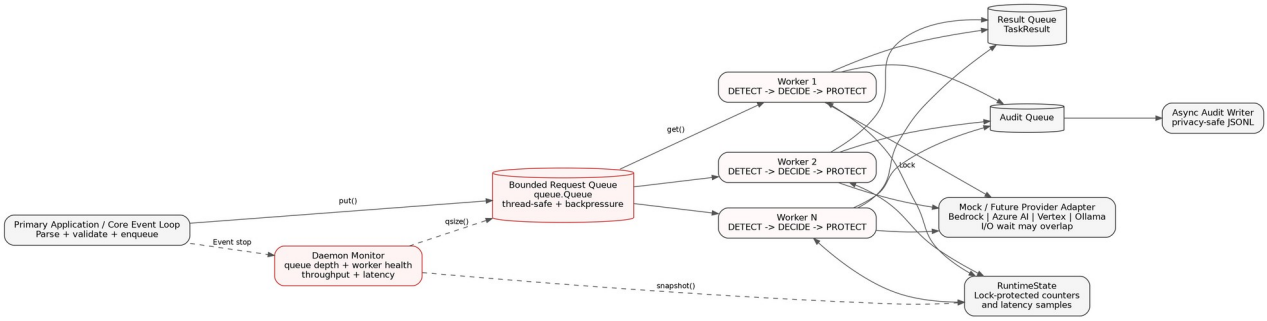


Figure 1. CipherMesh Assignment 2.1 concurrent runtime architecture.

The primary application thread remains responsible for input parsing and validation. Once an envelope is accepted, it places a WorkItem into a bounded queue. A worker obtains exactly one WorkItem, executes the unchanged privacy pipeline, optionally overlaps downstream/provider I/O, records metrics, returns a TaskResult, and submits a privacy-safe audit event. The monitor daemon observes the shared state and request queue without participating in request decisions.

3.1 Why a bounded queue

- It separates producers (the UI/API/event loop) from consumers (privacy workers).
- It provides natural backpressure; an unbounded queue can permit uncontrolled memory growth during a traffic spike.
- Python queue.Queue supplies the locking semantics required for multi-producer, multi-consumer exchange.
- task_done() and join() provide deterministic drain semantics before shutdown.

3.2 Why worker threads

A real LLM gateway spends substantial time waiting for network/provider responses, log I/O, or downstream services. Threads can overlap that I/O waiting while the main application continues to accept events. This design does not claim that Python threads accelerate CPU-bound regular-expression matching under the normal CPython GIL; the benchmark explicitly simulates I/O-bound provider latency to demonstrate the intended concurrency benefit.

4. Multi-threading / Worker Infrastructure

4.1 Worker-pool lifecycle

1. Create RuntimeState, bounded request Queue, result Queue, stop Event, AuditWriter, MonitorDaemon and N worker Thread objects.
2. Start the audit writer and all request workers, then start the monitor daemon.
3. Parse each incoming envelope on the primary thread. Rejected envelopes never enter the worker queue.
4. For each accepted request, create a WorkItem with an enqueue timestamp and place it in the queue.
5. A free worker dequeues the item, marks it in-flight, performs inspection/decision/protection, and produces a TaskResult.

6. The worker updates RuntimeState under a lock, posts the result, and enqueues an audit event without writing the log itself.
7. At shutdown, drain the queue, send one STOP sentinel per worker, join all workers, flush and join the audit writer, then signal the monitor Event.

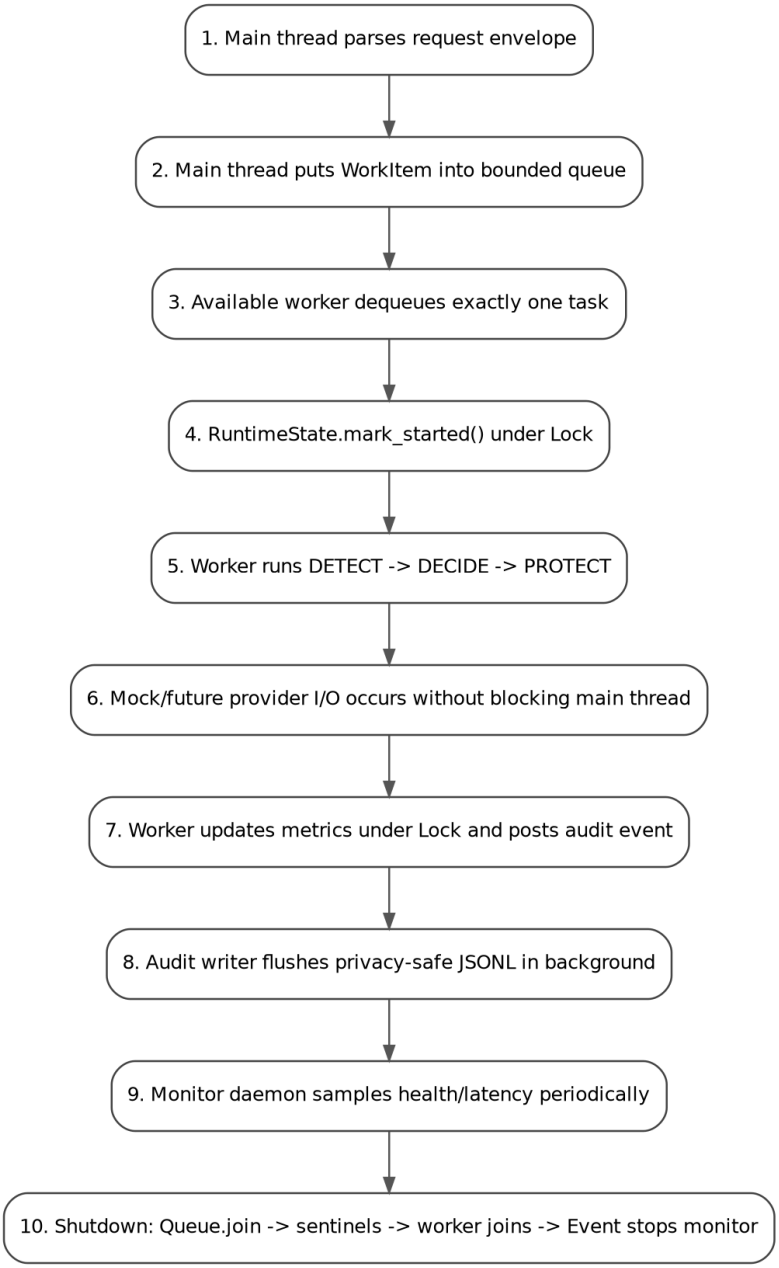


Figure 2. Deterministic request-to-shutdown lifecycle across threads.

4.2 Non-daemon workers vs daemon monitor

The request workers and audit writer are deliberately non-daemon threads. Accepted security decisions and audit evidence are transactional work that should finish or fail explicitly. The monitor is marked daemon=True because its role is observational and the process should not be kept alive solely for monitoring. Even so, CipherMesh does not rely on abrupt daemon termination: the monitor receives a threading.Event and is joined during clean shutdown.

Why this matters
Python documentation warns that daemon threads may be stopped abruptly at process shutdown. Critical request handling and audit flushing therefore remain non-daemonic; daemonization is reserved for the periodic monitor.

4.3 Backpressure and work ownership

Each WorkItem has one ownership transfer: producer -> queue -> exactly one worker. queue.Queue serializes get/put operations, and every successful get is paired with task_done() in a finally block. This prevents queue.join() from hanging because of a normal worker path. A finite queue_size also makes overload behavior explicit rather than hiding it.

5. Daemon Monitoring Functionality

MonitorDaemon is the Assignment 2.1 background monitor. It wakes at a configurable interval using stop_event.wait(interval) rather than a raw busy loop. The Event simultaneously supplies a responsive stop signal: setting it terminates the wait immediately and initiates final snapshot generation.

Metric / signal	Meaning	Why useful
queue_depth	WorkItems waiting for a worker	Reveals backpressure and worker saturation.
workers_alive	Number of request workers still alive	Detects worker loss or shutdown progress.
in_flight	Requests currently executing	Separates active work from queued work.
completed + decisions	Total finished plus ALLOW/REDACT/REVIEW/BLOCK counts	Operational and security outcome telemetry.
throughput_req_s	Completed requests divided by runtime	Simple capacity indicator.
avg / p95 service_ms	Worker processing + mock downstream I/O	Shows service-time stability.
avg / p95 queue_wait_ms	Delay between enqueue and worker start	Detects congestion.
avg / p95 end_to_end_ms	Queue wait + service time	User-visible request latency proxy.

In the verified run the monitor began with queue_depth=0 and 4 live workers, and ended in STOPPED state with queue_depth=0, workers_alive=0, completed=13 and in_flight=0. This demonstrates both periodic monitoring and clean termination.

6. Thread Safety & Concurrency Handling

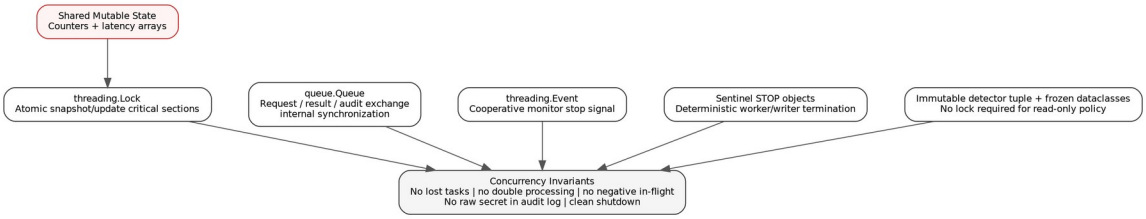


Figure 3. Synchronization primitives and the invariants they protect.

6.1 Shared-state strategy

Shared concern	Primitive / design	Invariant
Request transfer	queue.Queue	A request is handed to one worker; producers/consumers do not mutate a shared list directly.
Audit transfer	queue.Queue	Workers do not concurrently append to the same audit file.
Counters and latency arrays	threading.Lock inside RuntimeState	Compound updates and snapshots are consistent.
Monitor termination	threading.Event	Cooperative stop; no unsafe force-kill mechanism.
Worker / writer termination	Unique STOP sentinel objects	Each blocking get is released and each thread exits deterministically.
Detector policy	tuple + frozen dataclasses	Read-only policy can be shared without mutation locks.
Console output	ThreadSafeConsole + Lock	Complete log lines are not interleaved by multiple threads.

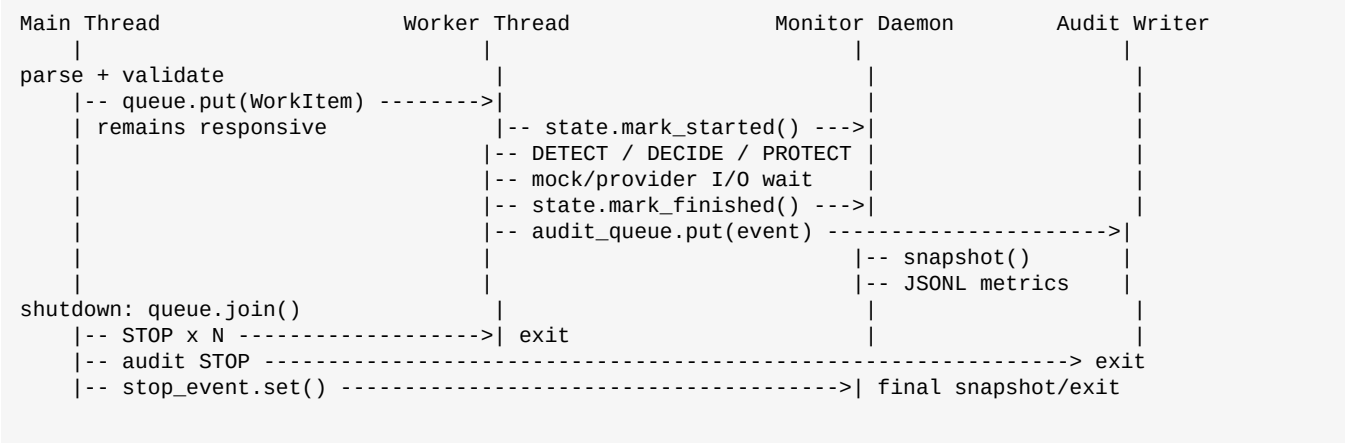
6.2 Why the GIL is not treated as a synchronization strategy

The implementation does not assume that the CPython Global Interpreter Lock makes application-level state safe. RuntimeState uses an explicit Lock for compound operations. This makes the intended critical sections visible, supports reasoning about correctness, and remains appropriate as Python evolves toward optional free-threaded builds.

6.3 Concurrency invariants

- completed never exceeds enqueued.
- in_flight returns to zero after the queue drains.
- Every accepted task is processed once and contributes to exactly one final decision counter.
- No raw original prompt is persisted to audit_events.jsonl; only prompt hash, finding metadata and an optional sanitized preview are stored.
- All non-daemon worker threads terminate before process completion.
- The monitor writes a final STOPPED snapshot with zero live workers after graceful shutdown.

7. Detailed Execution Workflow



The critical responsiveness property is visible in the execution log: the main thread emits CORE_EVENT_LOOP=RESPONSIVE while requests are still in-flight and queued. It only performs a blocking drain during controlled shutdown, not for every request.

8. Implementation Structure and Key Code

Class / function	Responsibility
LLMRequest.parse	Validate request envelope and provider before concurrency boundary.
inspect / decide	Assignment 1.2 privacy policy logic; stateless and worker-safe.
RuntimeState	Lock-protected metrics and snapshot API.
CipherMeshConcurrentAgent	Orchestration, lifecycle, queues and request worker pool.
AuditWriter	Dedicated asynchronous JSONL writer.
MonitorDaemon	Periodic health, queue, decision and latency monitoring.
process_request	Per-worker execution and latency measurement.
benchmark_concurrency.py	Controlled sequential/threaded I/O-bound benchmark.

8.1 Core worker loop excerpt

```
item = self.request_queue.get()
try:
    if item is STOP:
        return
    self.state.mark_started()
    result = process_request(...)
    self.state.mark_finished(result)
    self.result_queue.put(result)
    self.audit_writer.submit(self._audit_event(result))
finally:
    self.request_queue.task_done()
```

8.2 Monitor stop pattern

```
while not self.stop_event.wait(self.interval_s):
    record = self._sample("RUNNING")
    self._write(record)
# Event wakes the daemon immediately during shutdown.
```

8.3 Privacy-safe audit principle

The audit writer never receives the original prompt. The event contains request metadata, decision, finding types/count, sanitized preview when relevant, a truncated SHA-256 fingerprint, worker name and latency values. This preserves the project's original trust-boundary principle: observability must not create a second sensitive-data leak.

9. Performance and Latency Evaluation

9.1 Demonstration run

Configuration: 4 request workers, 0.05 s monitor interval, 120 ms explicit simulated provider I/O per accepted request, 13 accepted requests plus one rejected unsupported provider. The simulation is deliberate: it demonstrates the I/O-bound gateway case without contacting any paid or external service.

Metric	Verified value
Total wall time	503.702 ms
Accepted / completed	13 / 13
Rejected envelope	1
Outcome distribution	ALLOW 4 | REDACT 3 | REVIEW 2 | BLOCK 4
Average service latency	120.219 ms
P95 service latency	120.366 ms
Average queue wait	139.042 ms
P95 queue wait	241.085 ms
Average end-to-end latency	259.261 ms
P95 end-to-end latency	361.205 ms

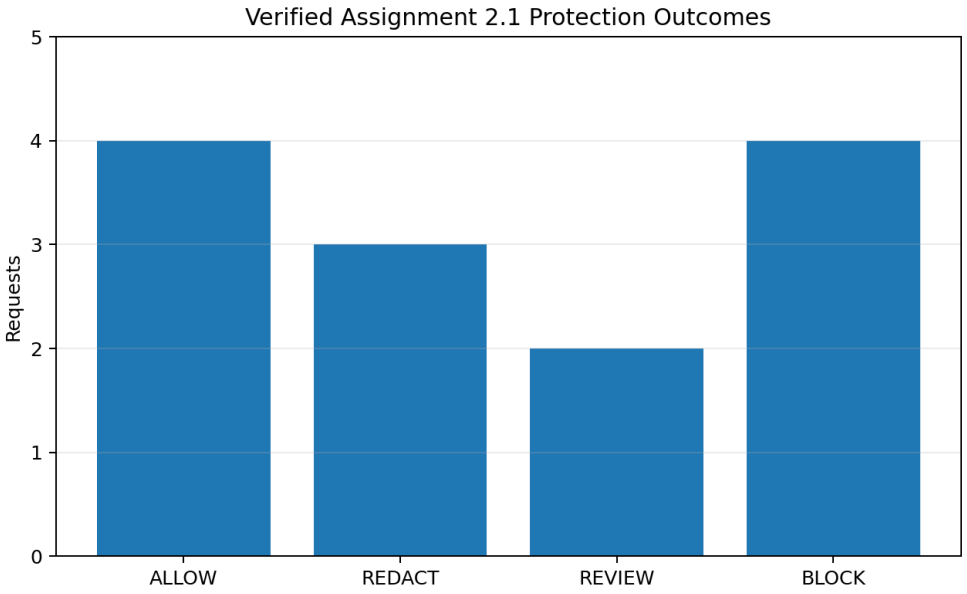


Figure 4. Protection outcomes from the verified Assignment 2.1 execution.

9.2 Sequential vs threaded benchmark

Benchmark configuration: 80 synthetic requests, each with 25 ms of simulated I/O-bound provider latency. The detector/policy remains real; only the downstream waiting time is simulated. Results below are measured from the included benchmark script in this runtime.

Mode	Wall ms	Req/s	Avg E2E ms	P95 E2E ms	Speedup
Sequential	2014.735	39.707	25.14	25.179	1.0x

Mode	Wall ms	Req/s	Avg E2E ms	P95 E2E ms	Speedup
Threaded (1 workers)	2025.889	39.489	1020.45	1915.182	0.994x
Threaded (2 workers)	1014.51	78.856	518.108	960.834	1.986x
Threaded (4 workers)	506.423	157.971	264.639	478.811	3.978x
Threaded (8 workers)	264.657	302.278	139.147	252.357	7.613x

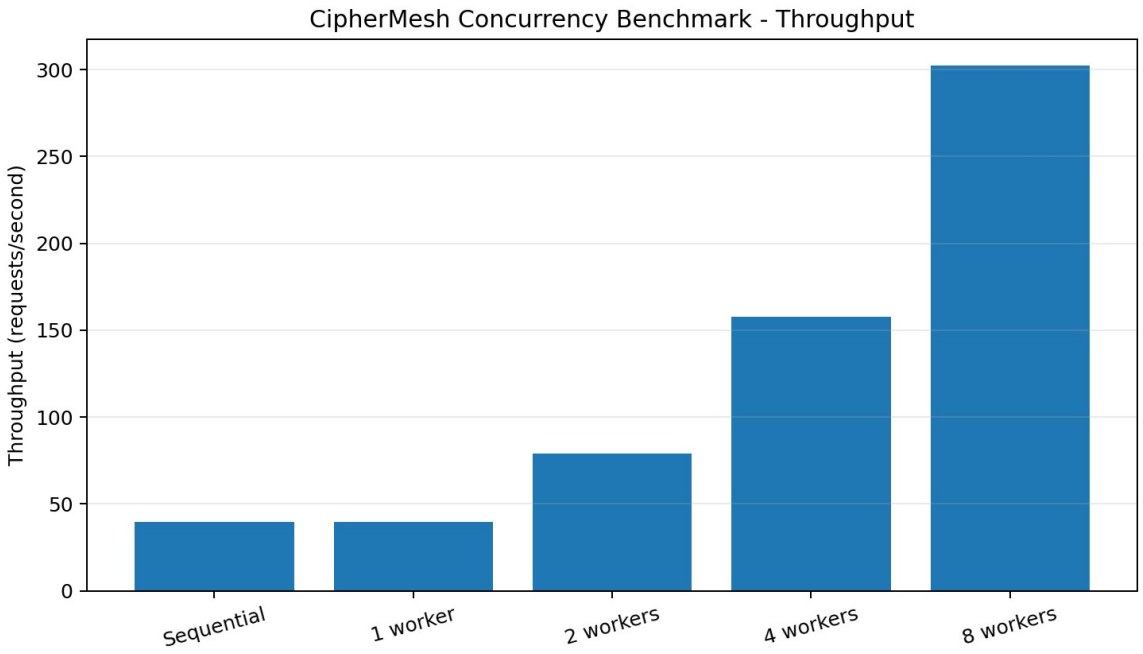


Figure 5. Throughput increases as I/O waits are overlapped by additional worker threads.

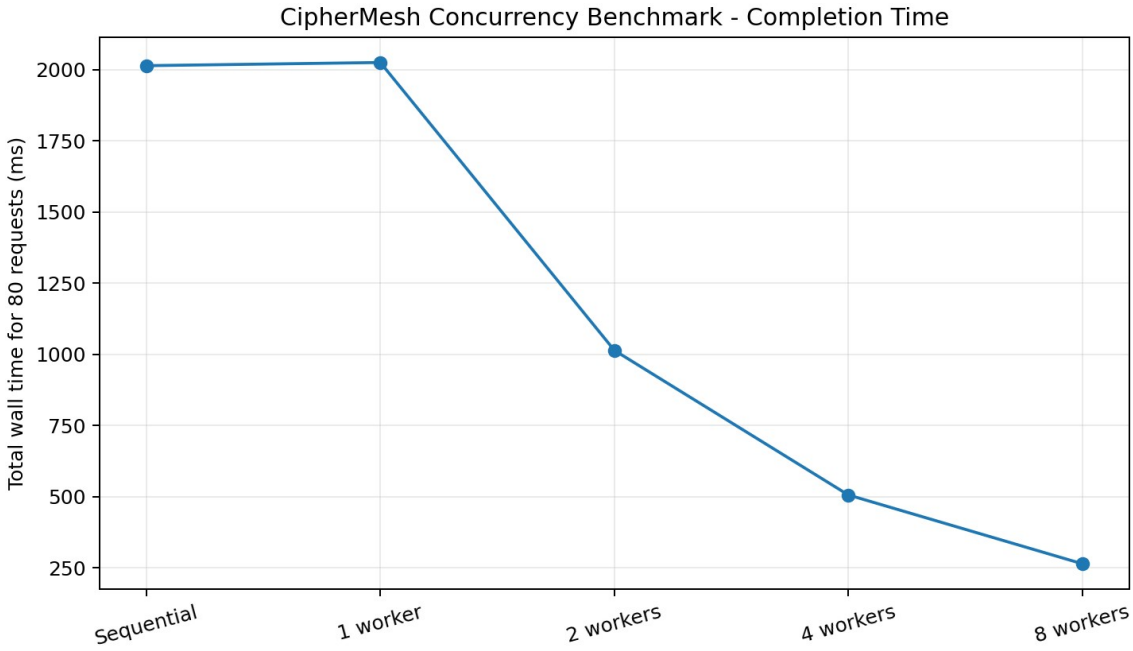


Figure 6. Total completion time for the 80-request benchmark.

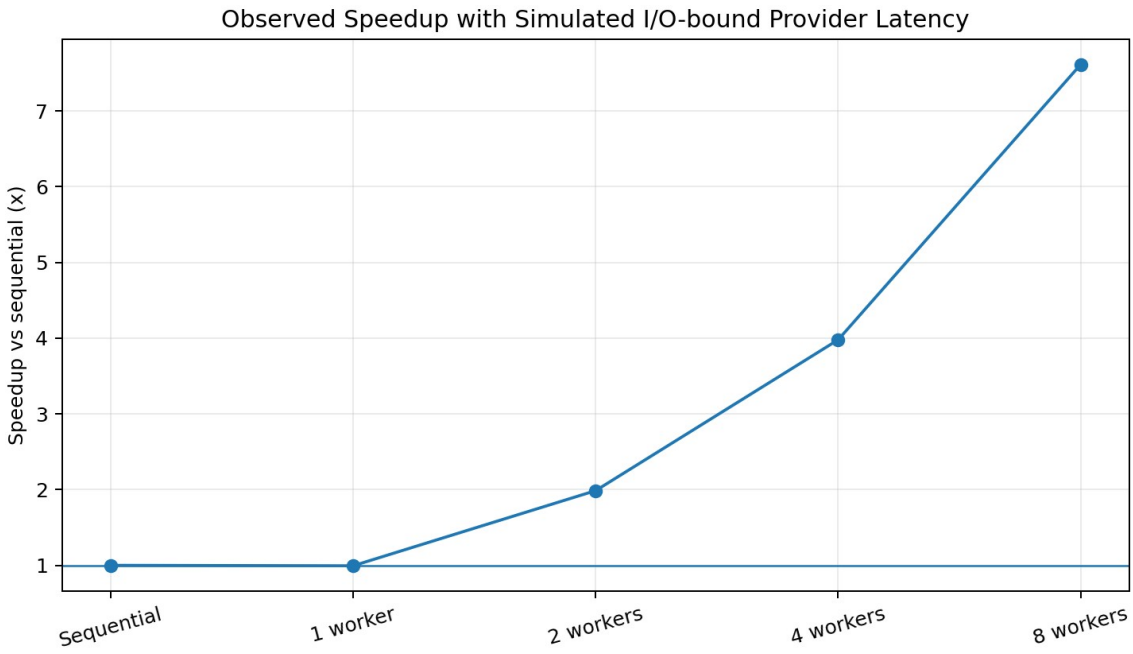


Figure 7. Observed speedup relative to the sequential baseline.

9.3 Interpretation

The sequential baseline completed 80 requests in 2014.735 ms (39.707 req/s). Four workers completed the same workload in 506.423 ms (157.971 req/s), a measured 3.978x speedup. Eight workers completed it in 264.657 ms (302.278 req/s), a 7.613x speedup. Service latency stayed close to the configured 25 ms because worker count changes how waits overlap, not the simulated service time itself.

The one-worker queued run has approximately the same total wall time as sequential but much higher average end-to-end latency because all 80 requests are enqueued immediately and then wait in line behind a single worker. This is a useful systems result: concurrency does not only affect total throughput; worker count and queue depth determine request waiting time and therefore perceived latency.

> **Performance claim boundary**
> These measurements demonstrate overlapping I/O-bound waiting. They do not claim that ordinary CPython threads produce equivalent speedup for CPU-bound regular-expression analysis, where the GIL limits simultaneous Python bytecode execution.

10. Verification, Testing and Evidence

Five standard-library unittest cases were executed successfully. Together they test both the inherited privacy semantics and the new concurrency infrastructure.

Test	Purpose	Result
test_assignment_1_2_policy_is_preserved	Critical secret still overrides lower-severity finding.	PASS
test_redaction_is_safe	Sensitive email/phone are removed from sanitized preview.	PASS
test_concurrent_exactly_once_accounting_and_shutdown	50 tasks complete exactly once; counters and worker shutdown are correct.	PASS

Test	Purpose	Result
test_privacy_safe_audit_never_contains_original_secret	Raw AWS-style secret is absent from audit JSONL.	PASS
test_monitor_daemon_writes_health_snapshots	Monitor produces STARTING/RUNNING/STOPPED telemetry.	PASS

Test execution completed 5 tests in approximately 0.235 seconds with status OK in the captured development run.

10.1 Evidence artifacts

- logs/console_execution.log - interleaved but line-safe proof that the main event loop remains responsive while workers execute.
- logs/execution_latency.log - per-request worker, decision, queue-wait, service and end-to-end latency.
- logs/monitor_metrics.jsonl - daemon health/latency snapshots over time.
- logs/audit_events.jsonl - asynchronous privacy-safe decision records.
- logs/benchmark_results.csv / .json / benchmark_summary.txt - reproducible performance evidence.

11. Mapping to Unit-2 Component Patterns

The assignment brief asks students to match structural patterns associated with Java Beans / ActiveX. CipherMesh implements this as a component-oriented Python architecture rather than claiming direct API equivalence. Each background service has a narrow public lifecycle and a well-defined data contract, which is the relevant structural lesson for this assignment.

Component-oriented idea	CipherMesh realization
Encapsulated reusable component	CipherMeshConcurrentAgent, AuditWriter, MonitorDaemon and RuntimeState are separate classes with focused responsibilities.
Lifecycle control	start(), submit_request(), wait_until_idle(), close() define explicit creation, operation and teardown.
Property/configuration surface	worker_count, queue_size, monitor_interval_s, simulate_provider_ms and log_dir are configurable constructor/CLI properties.
Event/message-driven interaction	Queue messages and Event signals connect components without direct unsynchronized mutation.
Observable state	MonitorDaemon reads RuntimeState snapshots and emits metrics records.
Component contract	WorkItem, TaskResult and audit dictionaries act as explicit exchanged data shapes.

> **Interpretation note**
> JavaBeans and ActiveX are component models, not Python threading primitives. The report therefore maps their course-level structural theme - modular, reusable, lifecycle-managed components - onto Python Thread/Queue/Event infrastructure.

12. Security and Privacy Properties

- Rejected envelopes are stopped before queue insertion, reducing unnecessary background work.

- Original prompts are not written to audit logs. A truncated SHA-256 fingerprint supports correlation without reproducing the prompt.
- BLOCK / REVIEW / REDACT paths persist only the already-sanitized preview.
- Detector definitions are immutable at runtime, avoiding accidental concurrent policy mutation in this assignment.
- The monitor observes counters and timing only; it does not inspect or persist prompt content.
- Thread-safe logging prevents accidental line interleaving that could corrupt evidence or make audits ambiguous.

12.1 Failure containment

The worker loop contains unexpected exceptions so a single malformed work item does not automatically terminate the whole process. The demonstration still treats such a worker error as visible operational evidence. In a production gateway, the next step would be structured error counters, retries only for idempotent downstream operations, circuit breakers, and external process supervision.

13. Limitations and Production Evolution

Current assignment boundary	Production evolution
Synthetic provider delay; no external API	Replace the mock wait with authenticated Bedrock/Azure/Vertex/Ollama adapters while preserving the same queue boundary.
Thread pool in one Python process	For CPU-heavy contextual classifiers, use process workers or dedicated inference services; retain threads for I/O-bound gateway work.
In-memory queues	Add durable broker semantics only when delivery guarantees require them.
Static immutable regex policy	Add signed/versioned policy reload and possibly Presidio/local contextual classifiers under a safe policy manager.
JSONL local audit	Move to append-only structured telemetry / SIEM export with retention and access controls.
CLI event loop	Wrap in FastAPI/OpenAI-compatible proxy so request acceptance remains asynchronous and provider-neutral.

13.1 Recommended next CipherMesh milestone

The strongest continuation after Assignment 2.1 is a FastAPI gateway that reuses this concurrency layer: HTTP request -> validation -> bounded work dispatch -> detector/policy engine -> provider adapter -> response scanner -> audit/metrics. The daemon monitor can then expose /health and /metrics-compatible state, while the same worker pool protects the API event loop from blocking provider calls.

14. Conclusion

Assignment 2.1 successfully transforms CipherMesh from a sequential simple-reflex prototype into a concurrency-aware agent runtime. The privacy semantics are preserved while request execution, audit persistence and monitoring are separated into safe background components. queue.Queue handles inter-thread work transfer, threading.Lock protects compound shared metrics, threading.Event controls cooperative daemon shutdown, and sentinels terminate blocking workers deterministically.

The captured run proves that the main event loop can remain responsive while four workers execute concurrent requests. The daemon records queue depth, worker health, decision counts and latency. The benchmark demonstrates near-linear improvement for the intended I/O-bound scenario up to the tested worker counts, while explicitly acknowledging the CPU-bound GIL boundary. This satisfies the assignment objective with implementation evidence rather than only a conceptual design.

Appendix A - Run Instructions

```
# 1. Run the Assignment 2.1 concurrent agent
python ciphermesh_concurrent_agent.py --workers 4 --monitor-interval 0.05 --simulate-provider-ms 120

# 2. Run sequential-vs-threaded benchmark
python benchmark_concurrency.py --requests 80 --io-ms 25 --workers 1 2 4 8

# 3. Run automated tests
python -m unittest discover -s tests -v
```

All code uses the Python standard library. Matplotlib/Graphviz were used only to generate report figures; they are not runtime dependencies of the CipherMesh agent.

Appendix B - Deliverable File Structure

```
CipherMesh_Assignment_2.1/
|-- ciphermesh_concurrent_agent.py
|-- mock_llm_requests_2_1.jsonl
|-- benchmark_concurrency.py
|-- README.md
|-- tests/
|   `-- test_concurrency.py
|-- logs/
|   |-- console_execution.log
|   |-- execution_latency.log
|   |-- monitor_metrics.jsonl
|   |-- audit_events.jsonl
|   |-- benchmark_results.csv
|   |-- benchmark_results.json
|   `-- benchmark_summary.txt
`-- docs/
    |-- Assignment_2.1_CipherMesh_Concurrent_Task_Daemonization_Report.docx
    |-- Assignment_2.1_CipherMesh_Concurrent_Task_Daemonization_Report.pdf
    `-- figures/ ...
```

Appendix C - Verified Log Excerpts

```
CipherMesh Concurrent Agent | STATUS=ONLINE | workers=4 monitor_daemon=ON audit_worker=ON
[MAIN] CORE_EVENT_LOOP=RESPONSIVE | completed=0/13 in_flight=3 queue=10
[MONITOR] queue=9 in_flight=4 completed=0 workers_alive=4 avg_service_ms=0.0
[cipher-worker-1] COMPLETE id=REQ-201 action=ALLOW ... service_ms=120.124
...
STREAM_COMPLETE | parsed=13 rejected=1 completed=13 ALLOW=4 REDACT=3 REVIEW=2 BLOCK=4
```

Final latency snapshot: avg_service_ms=120.219, p95_service_ms=120.366, avg_queue_wait_ms=139.042, p95_queue_wait_ms=241.085, avg_end_to_end_ms=259.261, p95_end_to_end_ms=361.205.

References

[1] Python Software Foundation. threading - Thread-based parallelism.
https://docs.python.org/3/library/threading.html

[2] Python Software Foundation. queue - A synchronized queue class.
https://docs.python.org/3/library/queue.html

[3] 21CSE525T Agent Technology - Assignment 2.1 brief supplied for this submission: Concurrent Task Daemonization using Threads and Workers.

[4] CipherMesh Assignment 1.1/1.2 project package and CipherMesh Red/White Presentation - prior project design and verified simple-reflex prototype used as the continuity baseline.

Access date for Python documentation: 2 September 2026.